

Workshop 1: Getting back into R (with answers)

Roos & Pinard

Last updated: 2024-09-24

Table of Contents

Workshop Structure	2
Learning Objectives	2
Instruction Orientation.....	2
Commenting guide.....	3
On using LLMs	5
Getting Started with R.....	5
Running Code	6
Workshop Instructions.....	6
Set working directory.....	6
Download the File	8
Load File into RStudio	8
Checking It Worked	8
Question Set One.....	9
Getting to Know the Data	10
Question Set Two	11
Data Detective.....	12
Question Set Three	14
A More Refined Detective.....	14
Question Set Four	15
Correcting Data	15
Question Set Five	16
Do it for real.....	17
Information about the dataset:	17
Question Set Six	19

Workshop Structure

The workshops on this course will have you learn an analytical workflow, focussing mainly on analysing data using linear models (and eventually generalised linear models). That is, you're going to learn how to go from getting a dataset to extracting evidence to answer your research question. Importantly though, this workflow is not unique to linear models (or GLMs). You would (or at least *should*) use this same general workflow, regardless the dataset, the scenario, or the analytical method you may find yourself using.

The general workflow we'll be using can be broken down into:

1. Formulate a research question.
2. Perform exploratory data analysis [Focus of today].
3. Identify any hidden assumptions.
4. Fit an appropriate model.
5. Diagnose the model and check assumptions.
6. Summarise the results.
7. Interpret and provide inferences.

For the BI3010 workshops we are generally going to skip Step 1 in the workflow for convenience sake, but in your own work Step 1 is key as every subsequent step in the workflow is dependent on your question. In your own work (e.g. your Honour's project) it's important to take the time to think carefully about what the question you want to answer is, how best to design an experiment or data collection protocol to get data suitable for answering it, which statistical method would be suitable for analysing the data, and what caveats and limitations come with all of these. Basically, don't take the omission of step 1 in this course as evidence that it's not important. It is.

Learning Objectives

1. (Re)familiarise yourself with the basics of R;
2. Use R to perform simple descriptive summaries of data and data processing;
3. Use the results of the EDA (Exploratory Data Analysis) to identify any issues in the data;
4. Begin thinking about the assumptions we make with a specific dataset even before we begin using statistics

All of these objectives link to useful skills for **initial, quick explorations** and screening of datasets. Because it's so important to ensure that the data we subsequently use in analysis is suitable for answering our question, the importance of such EDA cannot be overstated.

These skills will be used throughout the course.

Instruction Orientation

All instructions for the workshops are created in R using a tool called `RMarkdown`, which is then exported as Word documents. Within these instructions, code will either be inline

(meaning the code appears in the normal text line, like this) or in “chunks”. When in chunks, they will have the following colour coding:

```
# Comments are not read as code.  
# They are notes for ourselves and others.  
my_result <- a_function(argument1 = "character1",  
                        argument2 = variableA,  
                        argument3 = 1234,  
                        argument4 = c(TRUE, FALSE),  
                        argument5 = my_data)
```

- The light purple text preceded by “#” indicates comments. These will not be treated by R as code, so we are free to write whatever we want. See below for guidance on comment writing.
- The orange code (e.g. `a_function()`) denotes a function. Functions are blocks of reusable code, often written by someone else, to perform a specific task, for example, `mean()`. Underlying each function is a collection of “source” code to perform the specific task (if you want to see the source code for `mean()`, write `getAnywhere(mean.default())` in your console). Functions are always followed by an open parenthesis “(” and end with a closed parenthesis “)”.
- The pale blue code (e.g. `argument1 =`) indicates arguments - data, parameters, or choices that the function requires, or allows, to perform its purpose. You can think of arguments as the information required, or permitted, for the function to do its task. Multiple arguments are separated using commas.
- Green code, wrapped in quotation marks (e.g. `"character1"`), indicates strings or characters, which are types of data used to represent text.
- Black code (e.g. `variableA`, `my_result`, and `my_data`) indicates objects in the R environment. Generally, these are single values, columns of data, or even entire datasets.
- Finally, the maroon code (e.g. `TRUE` and `FALSE`) represents “Boolean” values. These are some of the simplest forms of data and are used to indicate, unsurprisingly, whether something is true or false (commonly used in functions to allow the user to say “Yes” or “No”).

If you’re not sure what a string is, or a function, don’t worry; there’s a short explainer further below.

Note that the colours used within the instructions use a custom colour which will not match the colours used by default in RStudio. This is perfectly fine. If you want to change the “theme” of RStudio, go to Tools > Global Options... > Appearance and play around with the different options.

Commenting guide

It’s good practice to make comments to yourself (and others) while writing code. Comments generally act as notes that help explain your general intent for a section of code, which in

three weeks' time can be an absolute lifesaver. They're also super useful for anyone else reading your code (and your commenting may be evaluated for some jobs).

There are many different opinions as to how to write good comments (see [here](#) for a particularly stern view), but a general principle I subscribe to is: aim to very concisely describe your general purpose for the next few lines of code. What are *you* trying to do, and not, what does *each line of code* do.

For example, I would write:

```
# Get R set up
library(ggplot2) # For plots
setwd("my/file/path")
my_data <- read.csv("data/sales_data.csv")

# Plot the data
ggplot(my_data, aes(x = country, y = total_sales)) +
  geom_bar()
```

And not:

```
# Load the ggplot2 package into the R environment so we can create plots later
# What happened to ggplot1?
library(ggplot2)

# Use the read.csv function to read the sales_data.csv file from the data directory
# and store the resulting data frame in a data frame called 'my_data'
# Remember that it's called my_data and not data. Calling it data made things break :(
my_data <- read.csv("data/sales_data.csv")
# my_data <- read.table("data/sales_data.csv") # DONT USE THIS IT DOESNT WORK
# mi_data <- read_csv("data/sales_data.csv") # ALSO DIDN'T WORK - DUNNO WHY

# Use ggplot to create a bar plot of the total sales by country for 2024.
# The x-axis represents the categories, and the y-axis represents the total sales.
ggplot(my_data, aes(x = country, y = total_sales)) + # using ggplot, take my_data, then plot country on the x and sales on the y
  geom_bar() # then have the data visualised as bar, using the structure described in the previous line of code
# When I see comments like this IRL, I die a little bit inside. Keep it clean and concise.
```

The latter is so overly descriptive that it becomes as much of a chore to read the comments as the actual code itself. In fact, I think the code does a better job of quickly and efficiently explaining its purpose than these comments do! Another way to think of how to best write comments is: Think of comments as post-it notes. You want them to be short and to act as a signal for sections that you want to dive deeper into (e.g. for debugging).

On using LLMs

LLMs (Large Language Models), or “AI”, are incredibly useful tools for programming. Whenever I am coding, I normally have an LLM open and ready for any requests I might have. Personally, I find them particularly useful for 1) doing boring or fiddly coding, 2) automating tasks, 3) solving one-off problems, and 4) simplifying and tidying my code (see [here](#) for a pragmatic discussion on the use of LLMs in coding). But using them comes with the requirement that you have to know enough about coding to know when the answer they’re giving you is useful or when they’re... not so useful. We’re not quite at the stage where “you don’t have to know how to write code to write code”, simply because a lot of the time the code LLMs produce simply doesn’t work. As in, it’s just nonsense. Especially as the complexity of the code increases beyond “basic”. But the benefit for you is that the learning curve of coding has definitely flattened.

Using LLMs effectively, however, is somewhat of a skill in itself, in the same way as knowing what code or error to Google before LLMs came about. As a tip, make your prompt as detailed as possible. Describe in simple terms what you want to achieve, include the relevant code you tried to use, include any errors, and if possible, what you think the problem might be. It’s also worth telling the LLM that you are working in R (so it doesn’t give you python code), and to mention any packages (more on these on Wednesday) you want to use, or, alternatively, if you want to use the base version of R.

From bitter experience, I’d strongly recommend having a separate script open to test the code provided by an LLM before replacing your own code. I’ve been bitten too many times by flawed and broken LLM code that destroys entire scripts because I immediately replaced my original code before testing the LLM’s version. Test first, then combine.

Getting Started with R

If you are comfortable with the basics of R, feel free to skip ahead to “Workshop Instructions”.

Before we begin the workshop, let’s make sure that you are set up to work with R and RStudio. If you’re working on your own machine, make sure you have R and RStudio installed. If you don’t, you can download R from [CRAN](#) and RStudio from [RStudio’s website](#). On university computers, these should already be installed.

Once you have R and RStudio set up, open RStudio. You should see a window with four main panes:

- **Source:** This is where you write your scripts.
- **Console:** This is where you can type and execute R code directly.
- **Environment/History:** This pane shows you the objects that are currently stored in R’s memory.
- **Files/Plots/Packages/Help/Viewer:** This is a multi-tabbed pane that allows you to navigate files, view plots, manage packages, and access documentation.

Running Code

There are a few ways to run R code:

- **Directly in the console:** You can type R commands directly into the Console pane and press Enter to run them. This is useful for very quick tests and calculations but avoid using this for project work (none of it is saved).
- **From a script:** Scripts are simple text files that contain R code. You can write multiple lines of code in a script and run them together or one by one. This is particularly useful for more complex analyses where you want to keep a record of your code.

To run code from a script, you can:

- Click the “Run” button at the top of the Source pane to run the selected line or code chunk.
- Press `Ctrl + Enter` (or `Cmd + Enter` on a Mac) to run the current line or selected code.

Workshop Instructions

In this workshop, we're going to load two datasets, starting with alligator bite strength, and carry out Exploratory Data Analysis (EDA) along with some simple Data Quality Assurance. Broadly, our aim in such cases is to identify *weird* things in the data that might indicate mistakes, such as data entry errors. In real-world analysis, this step is incredibly important because mistakes are easily made, especially during data entry. Imagine entering 5,000 nearly identical observations into an Excel document. People get bored, start thinking about other things, get mixed up about which observation they should be adding, among many other distractions. It's hard to stay super focused during mundane work for long periods. It would be unrealistic to expect or assume that mistakes would never happen, so we need to check for them. The problem is that if we don't spot these errors, they can significantly influence the results when it comes time to analyze the data.

The good news is that we don't need any complex statistics to check for these issues, and the R coding is *relatively* straightforward. The first step is to load the data into R.

We would like you to use RStudio, an Integrated Development Environment. Essentially, it's a program that makes interacting with R (the programming language) a much more pleasant experience. If you have not used RStudio before, please refer to the instructions on MyAberdeen titled “Getting Started with R and RStudio.”

Set working directory

If you haven't already created a folder for the course on your computer or your university drive, do so now. For example, you might create a folder in your H drive for the course, with subfolders for each of the workshops (e.g., `workshop1`), such that the file path to your working directory would be: `H:\BI3010\workshop1`.

Open RStudio and create a new R script (File > New File > R Script, or use the hotkeys Ctrl + Shift + N). Once open, you can now write code in the script. When you want to run that code, either press the "Run" button (top right of the main panel) or click anywhere in a line of code and use the hotkeys Ctrl + Enter.

With your script now open, to set the working directory to the folder you want to use for saving your work, you have two options:

Option 1:

Find the file path for your newly created folder (e.g. H: > BI3010 > workshop1). If you're not sure, an easy way to get this is by using File Explorer in Windows, go to your workshop1 folder, then select the address (or file path) from the navigation bar at the top (the equivalent of the URL bar in web browsers). It will probably look something like:

H:\BI3010\workshop1

In R, backslashes are used for specific purposes, so we want to change the backslashes to forward slashes. To get our file path ready for R, it should look like:

H:/BI3010/workshop1

If you're using a Mac, your working directory may be something like:
~/Documents/BI3010/workshop1.

We can now write some R code to tell R that this is the folder that we will be using in the current session, by setting the Working Directory using the function `setwd()`. The `setwd()` function just needs a single piece of information (technically called an "argument"), which is the file path. However, because the file path is a "string" (i.e., a bunch of letters, characters, and numbers), we need to ensure that we put the file path in quotation marks:

```
setwd("H:/BI3010/workshop1")  
# Or for Mac:  
setwd("~/Documents/BI3010/workshop1")
```

The benefit of doing it this way is that because we can include this code in our R script, we don't have to go through this pain of setting the working directory every time we come back to work on this script. It also means we can copy and paste this code when we get to subsequent workshops, just changing workshop1 with, for example, workshop2.

Option 2:

An easy option is to use the drop-down menu in RStudio: Session > Set Working Directory > Choose Directory...

Option 2 is easier if you're just doing a very quick project (i.e., an hour or two), but becomes a pain when you come back to the project, as you'll need to keep setting the working directory each time (and maybe make a mistake, causing more pain and frustration).

We'd advise using Option 1. It's a little more of a fuss the first time, but then you don't need to think about it again.

We can check if the working directory has been set correctly by asking R what the current directory is:

```
getwd() #to check which directory RStudio is using
```

Download the File

You can download the data from MyAberdeen. Make sure to save the file to your working directory.

On MyAberdeen in the Tuesday - Workshop folder within Week 1, you'll find a file called `alligator_bite.txt`. Either click on the file to download it or, if that isn't working, use the three dots to the right of the content area that has the file name to download it.

Once downloaded, make sure to move it to your `H:/BI3010/workshop1` folder (or whatever you chose to call your folders).

Load File into RStudio

Now that we've told R which folder contains our data (by using `setwd()`), we can load the data into R. Before we do so, make note of what type of file the data is saved as. The alligator bite strength data is saved as a text file, which we know because of the `.txt` extension. For data saved in this format, we use the `read.table()` function, where the arguments are predominantly what the file is called, and if the file contains headers.

To find out which arguments can be provided to a function you can type `?read.table` (or `?any_function_here`). If we do so, we can see there are lots of arguments we could use, but the crucial ones are `file` (which is the name of the file we want to load), and another called `header` which is logical (meaning either `TRUE` or `FALSE`) which tells R if the first row of our `alligator_bite.txt` file are the column names, which they are.

So to load our data, we use the following code:

```
ali <- read.table("alligator_bite.txt", header = TRUE)
```

Note that this is a single line of code on a single line of script. But there's nothing to stop us from spacing this out to make it easier to read, if we so wish:

```
ali <- read.table("alligator_bite.txt",  
                  header = TRUE)
```

Checking It Worked

There are a few things to make sure of when loading your data into R, but just experiment. The great thing about working in R is that nothing is permanent. If you break something, you can just run your code again. The only thing that is "permanent" is your script, hence why it's important to save often (and also to write comments to yourself). Added something new? Save and write a comment saying what it does. Want to remove something but not sure if it'll break your code? Put a `#` at the start of the line to "comment it out".

When loading your data, if you encounter an error, some common problems include:

- You may have set the wrong working directory.
- You may have misspelled one of the words (including using a capital letter instead of lower case).
- Your file may be missing a column heading.
- The file is not in the correct format (e.g., the file saved as a `.csv` instead of a `.txt`).
- The quotation marks are in the wrong location.
- You may have missed the final bracket `)` in the `read.table()` function.

To confirm that the file has been read properly, click the blue button next to the name in the Environment panel (normally in the upper right of RStudio).

If you want to visually inspect the data now that it's in R, you have a couple of options:

The following will show you the first six rows of your dataset (you can change how many rows are shown by using the `n` argument, e.g., `head(ali, n = 10)`):

```
head(ali)
```

Or you can open the entire dataset in a new tab by doing:

```
View(ali)
```

The output of `View()` has the benefit of being visually similar to Excel spreadsheets, with many of the same features such as allowing the user to sort and filter by columns.

Or you can output all rows into the console, simply by doing:

```
ali
```

However, we wouldn't recommend this last option as it's not particularly useful, given that it floods your console, making it hard to see anything of use.

Question Set One

Question 1: How many variables do we have in this dataset?

There are several ways to check this, you can use:

```
summary(ali)
ncol(ali) # Number of columns (i.e., variables) in the ali dataset
str(ali)  # Structure of the ali dataset
head(ali) # Displays the first few rows
View(ali) # Opens the dataset in a spreadsheet-like view
```

Question 2: What are the variable names?

*# bite and length
Above functions will give you this except for ncol()*

Question 3: Are the names capitalised?

```
# bite and length are both lower case
# A general tip - when you're creating your datasets, keep the variable/column
# names simple. You're going to be writing them out lots, so make typing easier
# for yourself by using short names that are all lower case.
# Definitely avoid names like "Alligator_Bite_Strength_PSI"
```

These questions may seem trivial, but honestly, they're the first questions we ask ourselves when we load a dataset into R for the first time.

Getting to Know the Data

Here's what we know about the data. The data contains `nrow(ali)` observations (sometimes referred to as samples) of alligator bite strength, measured in pounds per square inch. All `nrow(ali)` alligators were measured at Parker Island Gator Farm, Florida, USA. For each bite strength, the length of the alligator, from nose to tail, was measured and is provided in centimeters (to one decimal place).

Before we move ahead with the data, let's take a second to become familiar with alligator bite strength. Using your search engine of preference, do a quick search to find out the bite strength of alligators. Do the same for the length of an average alligator.

When we do exploratory data analysis, we're specifically looking for things that look *weird*. In order to know if something is *weird*, we need to know what "normal" is; hence why it's a good idea to have some sort of notion of what a "normal" alligator's bite strength and length may be.

Write down the average bite strength of an alligator, measured in pounds per square inch, and the average length of an alligator in centimeters (ideally as a comment in your script).

With that done, let's move on to understanding our data. For starters, let's check if the data is sensibly structured. In R, this means that numbers are numbers, words are characters (or sometimes factors), and so on. Basically, we want to make sure there aren't any bite measurements like 2174.94BIGBITE, which would mess us up by causing R to think that all bite values are characters and not numbers.

To do this quickly, we can use the `str()` function (short for "structure"):

```
str(ali)
```

We see that `ali` is a `data.frame` (great - it should be!), with 78 observations (or samples) and two variables. The `$` operator is used to extract specific variables from a dataframe, so `ali$bite` will give you just the bite variable from the `ali` dataset. This is why the second and third lines are preceded by `$`. From this, we can see that both the length and bite variables are being treated as numbers (indicated by `num`), and we get a handful of example values for both length and bite.

So far, we're not seeing anything unusual, which is great - it means we haven't identified any problems that we need to fix.

If we wanted to, we could extract the `length` variable from the dataset by doing:

```
ali$length
```

How would you extract the `bite` variable?

Let's use these extracted variables to calculate some summary statistics, starting with the mean (i.e. average). To do so, we use the `mean()` function. To find the average length of our alligators, we do:

```
mean(ali$length)
```

We can see that the average alligator in Parker Island Gator Farm is 374 cm long.

Compare this value with the mean length of alligators you found from a quick internet search. It might be a bit longer than expected, but this could be due to these alligators coming from a farm rather than the wild. Perhaps they're given lots of food to help them grow? While our numbers may not match exactly with online data, significant discrepancies could be worth investigating.

Similarly, calculate the average bite strength of our alligators and see if there's anything unusual there.

```
mean(ali$bite)
```

Assuming you found that alligator bite strength is roughly 2,000 pounds per square inch, our alligators appear to be biting with an extra 1,500 pounds per square inch *on average*. That seems like *a lot* more.

This is something that should make us sit up and pay attention. We don't know for sure if there's a problem, but we have reason to believe there *might* be one. It's now up to us to prove that there *isn't* a problem.

There are many descriptive techniques to check for anomalies in our data, but a convenient "short cut" is to use the `summary()` function. This function provides a generic summary of our entire dataset when we supply `ali` as the sole argument. Try this now and see if you can identify any issues with the data.

```
summary(ali)
```

Keep in mind, with EDA we're looking for anything *weird* in the data.

Question Set Two

Question 1. How small is the smallest alligator in the farm for which we have data, and what's the biggest?

```
# 153.5 cm to 2293.0 cm
# Multiple ways to check this
range(ali$length) # will give you the minimum and maximum
min(ali$length)   # will give you the minimum
max(ali$length)   # will give you the maximum
```

```
summary(ali)           # will give you this for all data in ali
summary(ali$length)    # will give you summary of just length
```

Question 2. What are the minimum and maximum values for bite strength?

```
# 832.9 PSI to 123632.0 PSI
range(ali$bite)
```

Question 3. Do any of these minimums or maximums seem strange?

```
# Yes! An alligator that's longer than a bus?!
# The same alligator, or a different one, that bites with
# 123,632 pounds per square inch, where the average is apparently
# about 2,000?!
```

Data Detective

Hopefully, you've identified at least one questionable value in our dataset. What should we do now? Throw out the entire dataset? (No... The answer is "No you do not throw out the dataset"—at least not yet...). Let's try to figure out what's going on, starting with finding the problem observations.

How do we do that?

Let's start by considering what we think a weird observation might look like. We can use the `summary(ali)` output to help us here. For instance, we know we have an alligator that was apparently 2293 cm long (23 m - that's longer than a bus!). How do we find that value in our dataset to see if there was anything else unusual about it?

This is where square brackets `[]` come into play. In R, square brackets are used for "indexing" to extract specific rows or columns from a dataframe. Think of square brackets as a way to point to or select parts of your data.

To use them effectively, first consider how you would find the information manually (e.g. in Excel):

1. We'd start by looking at the `ali` dataset.
2. We'd then skim through the `ali` dataset, looking for any rows where alligators are 2293 cm long.
3. Once we've found these rows, examine all the information related to those alligators.

How does that translate into R? Almost exactly—we just need to write those three steps using code.

1. We'd start by looking at the `ali` dataset.

```
ali
```

2. We'd then skim through the `ali` dataset, looking for any rows where alligators are 2293 cm long.

This is where we can use square brackets `[]` in conjunction with conditional statements. Conditional statements allow us to specify criteria like “when this condition is met, do *something*”.

In our case, the condition is finding alligators that are 2293 cm long. In R, we write this condition using double equal signs (`==`) to denote “equal to”. This is basically the same as your eyes going down the Excel column looking for 2293, noting any instances where it was true or not. For example:

```
ali$length == 2293.0
```

The output will be a list of 78 values, each being either TRUE or FALSE, corresponding to each row in the `ali` dataset. TRUE values indicate rows where the alligator is 2293 cm long, and FALSE values indicate rows where the alligator is not 2293 cm long (in R, “not equal to” is denoted as `!=`).

3. Once we’ve found these rows, examine all the information related to those alligators.

Once we’ve identified the rows we’re interested in (the simplest but least elegant way is to just count what row the TRUE is from in the above list), we can use square brackets to extract the relevant information. Square brackets in R let us select specific rows, columns, or individual values from datasets. When using square brackets, remember that the row number comes before the comma, and the column number comes after (e.g., `ali[row, column]`). If we omit either the row or column number, R will return all rows or all columns respectively. For instance, to view all the information (i.e., all columns) for the 20th row, you would use:

```
ali[20,]
```

To make life a bit easier for ourselves, we can use these square brackets in combination with the conditional statement to extract the entire row for the alligator that’s 2293 cm long. In this case, the TRUE and FALSE values act as indicators for which rows we want to display. If a value is TRUE, it means we want to see that row; if it’s FALSE, we don’t. We can achieve this by writing:

```
ali[ali$length == 2293,]
```

What do you make of this observation? We’re expecting `length` to be about 200 to 400 centimeters (2 to 4 m), but this alligator is apparently 10 times bigger than that. And `bite` is about 100 times bigger. Neither of the two values seem to have a decimal point (or at least R has given them values of zero after the decimal point).

Why might that be the case? Let’s see if we can find a “normal” observation to compare this giant alligator to.

Assuming the first observation is a normal one, use the square brackets to extract the first row of data from the `ali` dataset:

```
ali[1, ]
```

This command extracts the data from the first row of the `ali` dataset, allowing us to compare it with the giant alligator’s values.

Well, they're clearly different. The first alligator in our dataset looks pretty normal. Maybe a bit on the big side (just over 4 m long), but not weirdly so. There are only two possible explanations for this. Either there are giant alligators among us. Or, there has been a data entry mistake.

Question Set Three

Question 1. What may have occurred to have caused this apparent data entry mistake?

```
# This "giant" alligator is the result of a data entry mistake whereby the decimal points were missing for bite and length. The values should be 229.3 cm long and 1236.32 pounds per square inch. If you look at the raw data, you'll see it has no decimal points.
```

A More Refined Detective

The method we used to identify the data entry mistake is effective, but what if we want to be a more refined programmer and avoid repeatedly writing out 2293?

Instead, we can use a function that returns the maximum alligator length. The function to do just that is called `max()`:

```
# Get the maximum alligator length
max_length <- max(ali$length)

# Now use this value to find the problematic rows
ali[ali$length == max_length, ]
```

By storing the result of `max(ali$length)` in a variable (`max_length`), we can simplify our code and make it more readable. This way, if we need to refer to the maximum length multiple times, we only need to use the `max_length` variable. Within the `ali` dataset, this code extracts all columns (since we have not specified anything after the comma in the square brackets) for any rows where the length of an alligator is equal to the maximum length of any alligator.

We can also adapt the conditional statement to find rows based on different criteria. For example, if we wanted to extract all rows with alligators that were greater than the average length, we could do:

```
# Calculate the average alligator length
average_length <- mean(ali$length)

# Extract all rows where the alligator length is greater than the average
ali[ali$length > average_length, ]
```

In this case, `ali[ali$length > average_length, ]` will give us all columns for rows where the length of the alligator is greater than the calculated average length.

We now get data from more than one alligator, as more than one alligator meets our condition of being longer than average. We can also see the “data entry mistake alligator” using this method.

Let's refine our approach further by using a different function: `quantile()`. First, rerun the `summary(ali)` code to observe the quantiles (1st Qu. and 3rd Qu.) that it reports. These quantiles represent the values below which 25% of the data falls and above which 75% of the data falls.

We can retrieve these quantile values ourselves by doing:

```
quantile(ali$length, probs = c(0.25,0.75))
```

The `probs` argument in the `quantile()` function allows us to specify which probabilities we want to calculate quantiles for. In the previous example, we requested the 25% (0.25) and 75% (0.75) quantiles. Since we wanted to get these two values, we used the `c()` function, which combines (or "concatenates") multiple values into a single vector.

If we only need a single quantile, such as the 75th percentile, we can directly write `quantile(ali$length, probs = 0.75)`. In this case, there's no need to use `c()` because we're only asking for one value.

Question Set Four

Question 1. Using `quantile()` extract all rows in the `ali` dataset where `bite` values are above the 3rd quantile.

```
ali[ali$bite > quantile(ali$bite, probs = 0.75),]
```

Question 2. Repeat the above, but for bite values *less than or equal to* the 33% quantile. If you're unsure how to specify *less than or equal to* have a quick search online or use your preferred LLM.

```
ali[ali$bite <= quantile(ali$bite, probs = 0.33),]
```

Question 3. Using the tools from above, can you identify any other possible data entry mistakes caused by missing decimal points?

```
# None of the other observations appear to have this "missing decimal point" error
```

Correcting Data

So, we've identified a problem. What now? Do we delete the observation? Declare that *Sarcosuchus* are not extinct and are alive and well in Parker Island Gator Farm, Florida, USA (and have actually doubled in size)? Set up a Facebook group to share our concrete and irrefutable evidence of *Sarcosuchus* and start producing a film to share this wonderful news with the world? Obviously not. That'd be silly.

But we do need to do something. Row 20 of our dataset is clearly a mistake, but correcting it is a really sensitive topic. Part of the reason for that sensitivity is because a non-trivial number of scientists have been found to actually *fake* their data in order to further their own careers and prestige (e.g. see [here](#), [here](#), and most ironically [here](#) for examples).

We want to be better than these kinds of people and don't want to risk it looking like we're manipulating our data to serve our own selfish needs. So how do we resolve our issue? First, go back to the original data or the data provider. Not the dataset we downloaded (`alligator_bite.txt`) but the original values (hopefully) written in a notebook to double check what the original values were. Here, we'd contact Parker Island Gator Farm and ask them to confirm our suspicion that row 20 was simply missing decimal points.

In the interest of time, I'll confirm that it's missing decimal points so we can move on to fixing the problem. Specifically, `bite` should have two decimal points, and `length` should have one.

We have two options: fix the data in `alligator_bite.txt` and reload the data, or fix row 20 in R. It shouldn't come as a surprise that we *strongly* recommend doing this in R. Doing so makes our change clear and transparent - the change is right there in the code for all to see. Doing it in Excel means that we're kind of obscuring it, or at least we'd have to say "We had this weird observation, but we've totally sorted it out. Don't worry. *Trust us*".

So how do we fix row 20? Well, we use the skills from above: extracting, indexing, assigning, and simple math.

Let's start with `ali$bite`. We only want to change row 20. How do we select only row 20? `ali$bite[20]`. Note that we don't need to specify a column in the square brackets, because we're working with a single column; the `bite` column.

Next, we need to replace `ali$bite[20]` with an altered version of itself, so we would write:

```
ali$bite[20] <- ali$bite[20]
```

This will just replace the `bite` value from row 20 with itself. We're not actually doing anything here, but we're getting closer.

Next, we need to "add" back in the decimal point that was lost during data entry. `ali$bite[20]` is `1.23632^5`, but it should be `1236.32`. How do we shift the value two places to the right of the decimal point? Divide by 100, leaving us with:

```
# Resolving data entry issue with missing decimal point
ali$bite[20] <- ali$bite[20] / 100
```

Question Set Five

1. Resolve the data entry mistake for `ali$length`. Be sure to add a clear comment indicating that you have made this change, so that if anyone reviews your code, they understand the reason for the modification and don't suspect any foul play.

```
# Correcting missing decimal point
# Confirmed by Parker Island Gator Farm on 1/1/2020
ali$length[20] <- ali$length[20] / 10
```

2. Having made any changes to your data, it's always worthwhile checking it again to make sure you did not create a new mistake. Do so now using any techniques you feel are appropriate.

```
summary(ali)
```


Do it for real

Within MyAberdeen, there is an additional dataset called `animal_speed.txt`. Your objective here is to try and find all the data entry mistakes. This dataset is more challenging than the alligator one, with some issues being harder to detect. Additionally, the dataset is fairly large, making it impractical to manually search for errors. To help you out, here are some hints:

1. Use `unique()` to find and report unique entries in a column, which is useful for strings/characters/factors.
2. If there are NA values in a column, functions like `mean()` will return NA. To override this, you can specify `mean(variable, na.rm = TRUE)`, which will temporarily remove NAs from the `variable` before executing the function.
3. There are a total of five mistakes in the dataset.

As before, start by getting a feel for the data. What data do you have? What kind of values seem sensible, or what kind of values might seem *weird*?

Can you find them all? Once you have, take a look at the instructions with answers to see what the values should be and attempt to resolve them.

Information about the dataset:

- The weight of three species was measured (in grams).
- The species included Peregrine falcons, Saiga antelopes, and cheetahs.
- The speed of each individual was measured over a 50 m distance (in km per hour).
- A total of 500 individuals were measured, but these were not equally divided between the three species.
- Multiple people participated in data entry, and it is believed that not all were confident in how to do this.
- Physical copies of the data are available should any data need to be verified (contact one of the teaching staff if you need a data entry confirmed).

```
# Initial exploration of the dataset
# There are many ways you can identify the problems. Here's how we did it:
summary(fast)
str(fast)
unique(fast$species)
fast$weight[fast$weight < quantile(fast$weight, probs = 0.01)]

# Problems identified from initial exploration: ~~~~~
~~~~~
# 1. One weight observation is 0 g
# 2. One weight observation is 3 g
# 3. Species named "Peregrin Took" instead of "Peregrine Falcon"
# 4. There is an extremely high speed value (33346.0 km/h) for a Peregrine Fa
```

```

lcon
# 5. There is a missing value (NA) in the speed column
# ~~~~~

# Problem 1: Investigate the weight observation that is 0 g
# Identify rows where weight is 0
fast[fast$weight == 0,]

# Assume you confirm with the data provider and correct this value
# The correct weight should be 45514 g
# Note in the code below we don't have a "," after == 0? That's because we
# are working with a single column of data, the weight column, so there's
# no need to say "and all columns" by using the "," at the end.
fast$weight[fast$weight == 0] <- 45514

# Problem 2: Investigate the suspiciously low weight (3.0 grams)
fast[fast$weight == 3,]

# Correct the weight after confirmation with the data provider
fast$weight[fast$weight == 3] <- 47255

# Problem 3: Correct the species name "Peregrin Took" to "Peregrine Falcon"
# Identify the unique species names to find the typo
unique(fast$species)

# Correct the typo
fast$species[fast$species == "Peregrin Took"] <- "Peregrine Falcon"

# Characters can be converted into factors. In many cases it doesn't make a
# difference, but it can be harder to correct factors compared to characters.
# A general rule, leave strings as characters until you're confident they are
# "correct", and then convert to factors if you really want "factors" but
# in general there's rarely any downsides to leaving them as characters.
fast$species <- factor(fast$species)

# Problem 4: Investigate the extremely high speed value (35711 km/h)
fast[fast$speed == max(fast$speed, na.rm = TRUE),]

# Correct the speed after identifying it as a decimal point error
# This is actually quite tricky to do because of the NA value in speed
# Instead of using conditional statements, which would need to deal with the
# NA, we can just use the row number that we can see on the left of the above
# output
fast$speed[15] <- fast$speed[15] / 100

# Problem 5: Handling missing values (NA) in the speed column
# NA is used to indicate "Not Available" or simply "missing"
# Assume our data provider cannot confirm what this value should be
# meaning our only choice is to leave it as an NA

```

```
# Final Summary of the dataset after corrections  
summary(fast)
```

Question Set Six

If you haven't done so already, have a look at the instructions with the answers which contains what the actual values should be for four of the five data issues that you identified. Correct those values now (either using the code provided in the answer or create your own solution - there are multiple ways to correct the data here) then answer the following questions:

1. What is the median weight in this dataset?

```
median(fast$weight)
```

2. What is the median speed in this dataset?

```
median(fast$speed, na.rm = TRUE)
```

3. Which species is most common in this dataset?

```
# We can use the table() function to answer this but lots of different ways  
table(fast$species)
```